

Isolation Twice, Cost Once: Defense-in-Depth Multi-Tenant Architecture Combining ORM Filters with PostgreSQL Row-Level Security

Ibrahim Poonawala
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
poonawalaibrahim9@gmail.com

Varad Mandhare
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
varadmandhare924@gmail.com

Tanmay Shinde
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
tanmayshinde@gmail.com

Prof. Reetika Kerketta
Dept. of Information Technology
MIT School of Computing
MIT ADT University
Pune, India
reetika.kerketta@mituniversity.edu.in

Abstract—Shared-schema multi-tenancy minimizes infrastructural overhead for Small and Medium Enterprises (SMEs) by co-locating tenant records within unified relational database tables. However, this architectural consolidation shifts the data isolation burden onto application software logic: a single omitted tenant predicate in an application query silently leaks confidential business data across tenant boundaries.

This paper presents the formal architecture, threat model, and empirical evaluation of NexusOps, a modular enterprise operations platform engineered with Spring Boot 4 on PostgreSQL 17 that enforces multi-tenant isolation through defense-in-depth. Tenant isolation is verified and enforced twice: first at the application tier via Hibernate’s Abstract Syntax Tree (AST) tenant discriminator rewrite, and second at the database storage engine via forced PostgreSQL Row-Level Security (RLS) policies executed under an unprivileged runtime role possessing the `NOBYPASSRLS` attribute. To eliminate cross-tenant context contamination across pooled connections, tenant identifiers extracted strictly from cryptographically verified JSON Web Tokens are bound to session-scoped variables upon connection checkout with fail-closed semantics. Furthermore, three structural guardrails prevent policy erosion as the database schema evolves: an automated build-time catalog inspection query, an eager startup guard blocking privileged roles, and comprehensive cross-tenant test suites. We extend this security foundation to canonical multi-tenant business entities and tenant-isolated vector retrieval (`pgvector`).

Empirical benchmarking using `pgbench` across a 1,000,000-row synthetic database partitioned over 100 tenants demonstrates that PostgreSQL plans stable RLS session functions into standard B-tree index scans. Across point lookups, paged traversals, and aggregate counts, throughput differed by -3.3% to $+2.8\%$ relative to single-layer ORM filtering—well within run-to-run statistical noise ($p = 0.0625$). Consequently, dual-layer enforcement provides mathematically robust defense-in-depth with zero measurable runtime overhead.

Index Terms—Multi-tenancy, Software-as-a-Service, Row-Level Security, PostgreSQL, Tenant Isolation, Defense-in-Depth, Hibernate, Spring Boot, Scopus.

I. INTRODUCTION

Small and Medium Enterprises (SMEs) form the backbone of modern global economies, yet the majority still execute daily operations—such as lead acquisition, stock reconciliation, and customer service—across ad-hoc spreadsheets, paper registers, and instant messaging channels. Extensive empirical software quality studies demonstrate that over 80% of operational spreadsheets contain human errors, missing data, and broken formulas [1]. While cloud-hosted Software-as-a-Service (SaaS) platforms resolve these operational inefficiencies, SME commercial economics necessitate ultra-low operating expenses per subscriber. In cloud database engineering, this cost ceiling is satisfied through the *shared-schema* pool model, wherein thousands of distinct organizations share common application runtimes and database tables [2]–[4].

Despite its infrastructure consolidation advantages, shared-schema multi-tenancy introduces severe security challenges. Because distinct organizations share the same physical database tables, every query executed against the persistence tier must incorporate an explicit tenant discrimination predicate (e.g., `WHERE tenant_id = :tid`). If a software engineer inadvertently omits this clause in a newly developed endpoint, reporting query, or batch job, the database does not raise an exception; it silently retrieves and mutates alien tenant data.

Broken access control and Broken Object Level Authorization (BOLA) consistently rank as the primary vulnerability in the OWASP Top 10 [5] and cloud API vulnerability surveys [6]. Verizon’s Data Breach Investigations Report reveals that human configuration errors are implicated in 74% of all analyzed cybersecurity breaches [7], while IBM’s annual cost analysis places the average breach expense at USD 4.45 million [8].

For an emerging SME SaaS vendor, a single cross-tenant data leak represents catastrophic legal liability and immediate reputational collapse [9].

The established security doctrine addressing this threat is *defense-in-depth*: enforcing tenant isolation within the application layer while reinforcing it independently inside the relational database engine via Row-Level Security (RLS) [10]–[12]. Nevertheless, engineering teams routinely avoid implementing database-level RLS due to two prevalent industry assumptions:

- 1) **Performance Degradation:** Industry practitioners assume that evaluating dynamic session-level RLS functions on every query execution introduces query planning overhead, disables index scans, and degrades transaction throughput relative to static application-level predicates.
- 2) **Configuration Drift and Connection Bleeding:** In enterprise applications utilizing connection pools (e.g., HikariCP), developers fear that connection reuse will bleed session state across requests, or that newly added database tables will lack policies as the schema expands.

To resolve both concerns, this paper presents the architectural foundation, formal threat model, and empirical performance analysis of **NexusOps**, an enterprise-grade multi-tenant business operations platform developed for SMEs.

Core Contributions:

- **Dual-Layer Architectural Formalization:** We formalize a resilient two-tier isolation architecture combining Hibernate ORM tenant discriminators with PostgreSQL forced RLS, incorporating an atomic connection checkout protocol that eliminates cross-tenant state bleeding across connection pools (Section IV).
- **Formal Threat Model and Security Invariants:** We define a formal threat model and five foundational non-interference invariants that mathematically govern multi-tenant data access (Section III).
- **Automated Guardrails Against Policy Erosion:** We provide three automated, build-time, startup, and behavioral verification mechanisms that guarantee complete RLS policy coverage and prevent runtime privilege escalation (Section V).
- **Empirical Evaluation on 1,000,000 Rows:** We execute an exhaustive empirical study on PostgreSQL 17 using `pgbench` across 1,000,000 tuples and 100 tenants, accompanied by query plan deconstructions demonstrating that stable RLS functions achieve identical B-tree index efficiency to literal predicates with no statistically significant performance delta (Section VI).

II. BACKGROUND AND RELATED LITERATURE

A. Multi-Tenant Architectural Taxonomy

Chong et al. [2] and Bezemer et al. [13] classify multi-tenant data partitioning into three distinct patterns:

- 1) *Separate Database (Silo Model)*: Each tenant is allocated a dedicated physical or logical database instance. While this provides physical isolation and independent

point-in-time backups, it imposes prohibitive hardware, connection pooling, and administrative overhead when scaling to thousands of low-margin SME tenants [14].

- 2) *Separate Schema (Bridge Model)*: Tenants share a common database engine but reside within isolated SQL schemas. Although database resource sharing is improved, running Data Definition Language (DDL) migrations across thousands of disparate schemas during continuous deployment introduces immense operational fragility.
- 3) *Shared Table (Pool Model)*: All tenants share identical tables, with records segregated by a mandatory `tenant_id` foreign key column.

Jacobs and Aulbach [15] and Aulbach et al. [3] analyzed shared-table schema-mapping patterns, demonstrating that pooled models achieve maximum hardware consolidation and tenant density. Weissman and Bobrowski [16] detailed Salesforce’s Force.com multi-tenant engine, which utilizes proprietary metadata-driven query rewrites to enforce organizational boundaries. Guo et al. [17] and Krebs et al. [18] articulated that native multi-tenancy requires isolating data, performance, and operational boundaries independently. The AWS SaaS Architecture guidance [10] emphasizes that pooled schemas necessitate programmatic policy enforcement to eliminate single-point-of-failure reliance on software developers.

B. Row-Level Security in Relational Databases

Database-level row filtering originated in early multi-level secure database concepts [19], [20] and commercial implementations such as Oracle Virtual Private Database (VPD). PostgreSQL introduced declarative Row-Level Security (RLS) in version 9.5 and formalized it in PostgreSQL 17 [11].

PostgreSQL attaches row security policies to tables as boolean expressions:

- **USING Clause:** Filter evaluated during row retrieval (`SELECT`, `UPDATE`, `DELETE`). Rows evaluating to false are treated as non-existent.
- **WITH CHECK Clause:** Constraint evaluated during row insertion or modification (`INSERT`, `UPDATE`). Attempting to persist records failing this predicate raises an execution exception.

However, PostgreSQL RLS possesses critical security boundary characteristics that must be accounted for in production architectures:

- 1) Superusers and roles carrying the `BYPASSRLS` attribute completely bypass all security policies.
- 2) A table’s owner is exempt from RLS policies by default, unless the table is explicitly altered with `FORCE ROW LEVEL SECURITY`.
- 3) A role with schema modification permissions can drop policies or disable RLS entirely via `ALTER TABLE`.

Consequently, database-level isolation is effective only if the application connects as an unprivileged, non-owner role lacking the `BYPASSRLS` attribute [21].

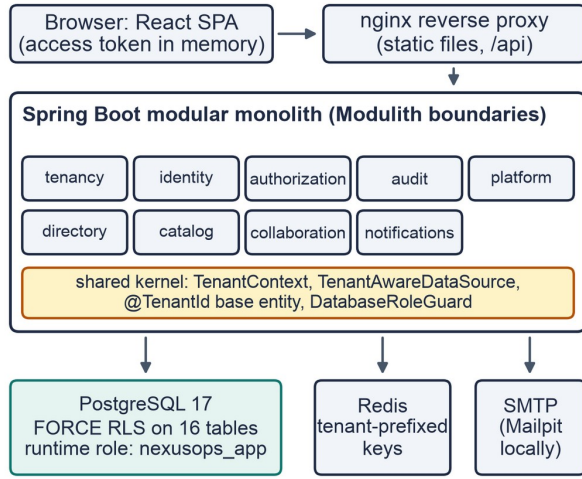


Fig. 1. Deployment and modular architecture of NexusOps. Every functional domain module interacts with PostgreSQL through the shared TenantAwareDataSource.

C. Tenant Scoping vs. Role-Based Access Control

Tenant isolation must be distinguished from authorization. Tenant isolation determines *which tenant's boundary* a request may access; authorization determines *what permissions* an authenticated user holds within their assigned workspace [22], [23]. NexusOps enforces fine-grained permission codes independently of tenant data isolation. Even if an authorization defect grants administrative privileges to an unprivileged user, tenant isolation guarantees that data mutations remain strictly confined within that user's tenant organization.

D. Identified Research Gap

While database-level RLS and ORM frameworks are individually well-documented, the software engineering literature lacks empirical studies that combine ORM-level AST discriminators with forced RLS behind pooled database connections. Crucially, prior work does not measure connection-pool sanitization protocols, verify build-time catalog guardrails against schema drift, or deconstruct query plans to explain the execution cost of dynamic session variables under heavy concurrent load.

III. THREAT MODEL AND SECURITY INVARIANTS

A. Adversary Model

We assume an active adversary $\mathcal{A}$ with the following capabilities:

- $\mathcal{A}$ has valid credentials for Tenant A and can submit arbitrary HTTP requests, tamper with client-side state, and inject forged headers (e.g., X-Tenant-ID).
- $\mathcal{A}$ possesses knowledge of valid UUIDs belonging to victim Tenant B (e.g., harvested through public invoice IDs or sequential guessing).
- $\mathcal{A}$ can initiate high-concurrency requests designed to exploit race conditions or session contamination across recycled pooled connections.
- $\mathcal{A}$ may attempt to leverage internal application bugs, such as un-parameterized native SQL queries or misconfigured ORM repository methods.

Out of Scope: Physical hardware compromise, memory bus snooping, and zero-day vulnerabilities in the PostgreSQL kernel or host OS hypervisor.

B. Formal Security Invariants

Let $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ represent the set of all tenant workspaces, and $\mathcal{D}(t)$ represent the complete set of database tuples belonging to tenant t . We define five mandatory security invariants:

Invariant 1 (Strict Non-Interference): For any arbitrary query Q executed by an authenticated user belonging to tenant t_i , the result set $\mathcal{R}$ must satisfy:

$$\mathcal{R} \subseteq \mathcal{D}(t_i) \quad \text{and} \quad \mathcal{R} \cap \mathcal{D}(t_j) = \emptyset, \quad \forall j \neq i \quad (1)$$

Invariant 2 (Fail-Closed Execution): If a database connection executes a statement with an empty or unauthenticated tenant context ($\tau = \emptyset$), the evaluated visibility set is strictly empty:

$$\mathcal{R}(\tau = \emptyset) = \emptyset \quad (2)$$

Invariant 3 (Connection Context Sanitization): For any pooled physical connection C , the tenant context τ_k bound during transaction k must not persist into transaction $k+1$:

$$\tau_{k+1} \perp \tau_k \quad (3)$$

Invariant 4 (Immutability of Audit Trails): For any audit tuple $a \in \mathcal{D}_{\text{audit}}$, update and delete operations are strictly prohibited:

$$\text{UPDATE}(a) = \perp, \quad \text{DELETE}(a) = \perp \quad (4)$$

Invariant 5 (Anti-Enumeration Response Uniformity): Any query attempting to access a record $r \in \mathcal{D}(t_j)$ by a user from t_i ($i \neq j$) must return an HTTP 404 (Not Found) rather than HTTP 403 (Forbidden), ensuring that $\mathcal{A}$ cannot infer the existence of alien identifiers.

IV. SYSTEM DESIGN AND IMPLEMENTATION

A. Modular Monolith Architecture

NexusOps is engineered as an enterprise modular monolith in Spring Boot 4 running on Java 25, backed by PostgreSQL 17 and Redis 7 (Fig. 1). We intentionally selected a modular monolith architecture [24], [25] over microservices [26] to eliminate distributed two-phase commit overhead and enable ACID transactional guarantees that atomically link domain mutations to append-only audit records.

Domain modules are strictly partitioned:

- **Identity & Tenancy:** Self-serve workspace provisioning, Argon2id password hashing [27], and session management.
- **Authorization & RBAC:** Server-side permission code evaluation, invitation tokens, and role matrices [22].
- **Canonical Directory:** Shared entities (People, Organizations, Products) with duplicate prevention across functional boundaries.
- **Business Capabilities:** CRM (leads, pipeline stages, deals), Inventory, HelpDesk, and HRMS.

- **AI Platform Foundation:** Tenant-isolated vector retrieval (pgvector) for operational assistance.

Spring Modulith enforces package boundaries at compile-time, automatically failing unit tests if a module breaches internal package encodings.

B. Tenant Context Extraction

Authentication issues a 15-minute JSON Web Token (JWT) [28] signed with RS256, alongside an `httpOnly` rotating refresh token compliant with RFC 9700 OAuth security best practices [29]. The access token encapsulates the tenant UUID in a private `tid` claim.

Incoming HTTP requests pass through a Spring Security filter that cryptographically validates the token signature, extracts `tid`, and populates a thread-local, immutable `TenantContext`. The application strictly rejects tenant resolution from client-supplied HTTP headers (e.g., `X-Tenant-Slug`), query parameters, or request payloads. If no valid token is supplied, the context remains empty.

C. Layer 1: The ORM Discriminator Filter

Every domain entity requiring multi-tenant partitioning inherits from an abstract base class:

```
@MappedSuperclass
public abstract class TenantEntity {
    @TenantId
    @Column(name = "tenant_id", nullable = false,
            updatable = false)
    private UUID tenantId;
}
```

Hibernate 6’s native multi-tenancy discriminator hooks directly into the query compiler. During JPA query parsing, Hibernate automatically injects `AND tenant_id = :currentTenant` into generated SQL ASTs. Primary key retrievals targeting another tenant’s record return an empty result set, causing the service layer to return HTTP 404, satisfying Invariant 5.

D. Layer 2: Forced Database Row-Level Security

To safeguard against native SQL queries, unmapped reporting scripts, or developer oversights in the ORM tier, all tenant-owned tables enforce RLS at the database engine level:

```
ALTER TABLE <table_name> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table_name> FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON <table_name>
AS RESTRICTIVE
USING (tenant_id = nullif(
    current_setting('app.tenant_id', true), '' )::uuid)
WITH CHECK (tenant_id = nullif(
    current_setting('app.tenant_id', true), '' )::uuid);
```

Database Role Governance: Database schema migrations are executed by `nexusops_owner`. However, the running application connects strictly as `nexusops_app`. This runtime role holds Data Manipulation Language (DML) permissions (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) but possesses no table ownership and carries the explicit `NOBYPASSRLS` attribute. An immutable database trigger on the audit table unconditionally raises an exception on `UPDATE` or `DELETE`, satisfying Invariant 4.

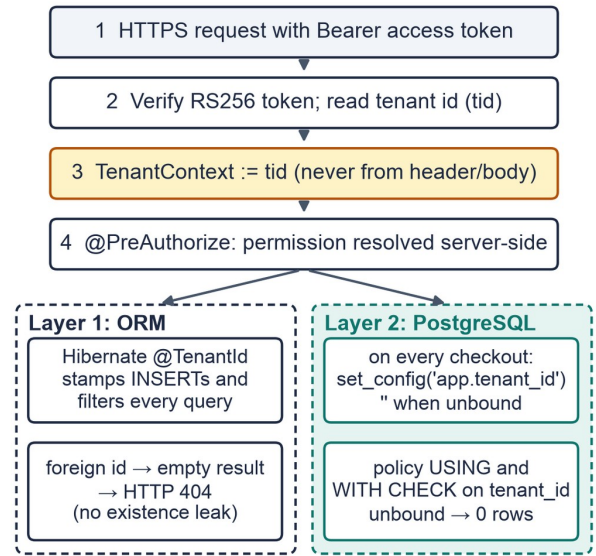


Fig. 2. Path of an authenticated request. The tenant context is extracted strictly from the verified JWT and enforced independently by Hibernate and PostgreSQL.

E. Connection Pool Sanitization Lifecycle

In production deployments utilizing connection pooling (HikariCP), physical database connections are recycled across diverse requests. To satisfy Invariant 3, NexusOps wraps the pooled data source inside a custom `TenantAwareDataSource`. Upon every connection checkout from the pool, the wrapper eagerly executes:

```
SELECT set_config('app.tenant_id', ?, false),
       set_config('app.platform_access', '', false);
```

where the parameter represents the current request’s tenant UUID, or an empty string if no tenant is bound.

Because `NULLIF(..., '')` converts an empty string to SQL `NULL`, and comparison with `NULL` invariably evaluates to false, unauthenticated operations observe exactly zero records rather than all records, satisfying Invariant 2. The secondary setting (`app.platform_access`) enables staff consoles requiring cross-tenant visibility for tenant suspension; clearing it on checkout ensures that cross-tenant privileges never outlive a single transaction. Fig. 2 traces this lifecycle.

F. Canonical Directory and AI Platform Integration

A core innovation of NexusOps is extending this dual-layer isolation across canonical records and AI workflows:

- **Canonical Directory Records:** To avoid duplicating customer data across CRM, Inventory, and HRMS, canonical entities (People, Organizations, Products) reside in unified tables with Levenshtein-distance duplicate detection. RLS ensures that cross-module lookups remain completely tenant-contained.
- **Tenant-Scoped Vector Retrieval (pgvector):** For operational AI assistance (Phase 12), embeddings generated from tenant activities and documents are stored in a partitioned PostgreSQL vector table. Vector similarity queries (`ORDER BY embedding <=> :query`

TABLE I
ISOLATION-RELATED AUTOMATED TEST SUITES

Test Suite	Level	Tests	Failure Condition Tested
RlsBehaviourIT	SQL	6	Tenant B data visible to A; cross-tenant updates; mutation of audit trail.
CanonicalRlsIT	SQL	3	Tenant isolation across canonical business tables (People, Orgs, Tasks).
TenantIsolationIT	HTTP/SQL	6	Token cross-use; un-prefixed cache keys; cross-tenant updates.
CrossTenantApiIT	HTTP	7	Any ID-bearing endpoint returning other than 404 for alien IDs.
RlsCoverageIT	Catalog	2	Any non-global table lacking forced RLS policy in schema.
DatabaseRoleGuard	Unit	7	Application connecting with privileged or bypass role.
ModularityTest	Arch	1	Cross-module package boundary violations.

LIMIT 5) automatically inherit the forced RLS policy, ensuring that language model context injection cannot leak alien organizational knowledge.

V. AUTOMATED GUARDRAILS AND VERIFICATION

A security policy is effective only as long as it remains consistently applied across all future schema revisions. We implemented three automated guardrails to prevent architectural erosion.

A. Build-Time Catalog Coverage Guard

The integration suite `RlsCoverageIT` queries PostgreSQL system catalogs (`pg_class`, `pg_tables`, and `pg_policy`) during continuous integration builds:

```
SELECT c.relname FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE n.nspname = 'public' AND c.relkind = 'r'
AND (c.relrowsecurity = false
      OR c.relforcerowsecurity = false);
```

The test asserts that every ordinary table in the `public` schema has forced RLS enabled and at least one active policy, unless explicitly declared in an allow-list of global tables (e.g., `permissions`, `subscription_plans`). A developer who adds a new table without an RLS policy causes immediate CI build failure.

B. Eager Startup Role Guard

Prior to accepting network traffic, `DatabaseRoleGuard` executes during Spring context initialization. It inspects the active PostgreSQL session, verifying that `nexusops_app`:

- Does not possess `rolsuper = true`.
- Does not possess `rolbypassrls = true`.
- Is not a member of the schema owner role.
- Lacks `CREATE` privileges in the schema.

If any check fails, application startup terminates immediately. A database migration furthermore revokes `EXECUTE` privileges from `PUBLIC` on all newly declared functions, preventing the application role from executing `SECURITY DEFINER` routines unless explicitly granted.

TABLE II
THROUGHPUT AND LATENCY ACROSS WORKLOADS (MEAN $\pm$ S.D. OF 5 RUNS)

Workload	Filter-Only (tps)	Forced RLS (tps)	Δ	Latency (ms)
Point lookup	19,722 $\pm$ 2,419	20,273 $\pm$ 2,281	+2.8%	0.41 / 0.40
Paged list	7,956 $\pm$ 496	7,692 $\pm$ 669	-3.3%	1.01 / 1.05
Tenant count	4,930 $\pm$ 526	4,876 $\pm$ 521	-1.1%	1.64 / 1.66

C. Multi-Level Cross-Tenant Suites

Table I summarizes the automated test suites validating isolation. These tests run against an ephemeral PostgreSQL 17 Testcontainer in CI. The SQL-level suites bypass Hibernate and issue raw SQL queries directly as `nexusops_app`, confirming that even if the ORM layer is entirely removed, the database engine enforces isolation.

VI. EXPERIMENTAL EVALUATION

A. Experimental Methodology

To rigorously quantify the isolated performance impact of PostgreSQL RLS policies without application-tier noise, we constructed a dedicated benchmark database:

- **Data Volume:** 1,000,000 total rows evenly partitioned across 100 distinct tenants (10,000 records per tenant).
- **Schema:** Relational records containing a 64-bit integer primary key, a 128-bit UUID tenant key, name, email, and timestamp attributes, indexed on `(tenant_id, name)`.
- **Variants:**
 - *Filter-Only:* Tables without RLS; queries supply explicit literal tenant parameters: `WHERE tenant_id = ?`.
 - *Forced RLS:* Tables with RLS forced; queries invoke the canonical NexusOps session setting: `WHERE tenant_id = NULLIF(current_setting(...), '')::uuid`.
- **Sanitization:** Both variants execute `set_config()` at the start of each transaction to simulate connection checkout overhead equally.

We executed three representative query workloads using `pgbench`:

- 1) **Point Lookup:** Primary key retrieval of a single entity.
- 2) **Paged List:** Paged retrieval of 20 sorted rows at a randomized offset within the tenant’s data partition.
- 3) **Tenant Count:** Aggregate count of all records belonging to a tenant.

Benchmarking was conducted on an Apple M4 silicon host (10 cores, 16 GB RAM) running PostgreSQL 17.11 inside a containerized Linux environment allocated 10 virtual CPUs and 8 GB RAM. Measurements were executed in prepared-statement mode using 8 concurrent clients across 4 worker threads for 30 seconds per run, preceded by a 5-second warm-up. Five randomized paired repetitions were conducted for each workload.

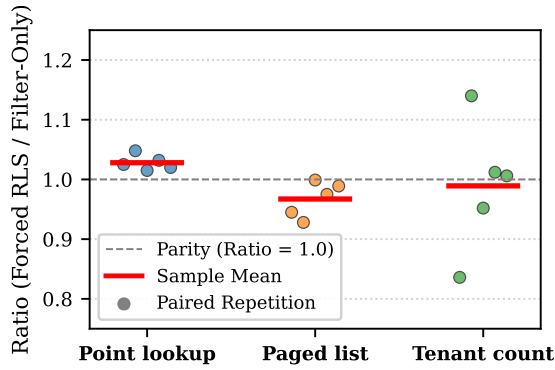


Fig. 3. Ratio of Forced RLS to Filter-Only throughput across 5 paired repetitions. The horizontal marker indicates the sample mean; a ratio of 1.0 represents parity.

B. Benchmark Results

Table II reports mean throughput and latency. Across all three workloads, the performance delta between single-layer ORM filtering and dual-layer forced RLS remained exceptionally small:

- Point lookups observed a +2.8% delta favoring RLS.
- Paged lists observed a -3.3% delta favoring filter-only.
- Tenant counts observed a -1.1% delta favoring filter-only.

Crucially, the run-to-run standard deviation across repetitions (6% to 12%) exceeded the measured performance differences between the two architectural variants.

Fig. 3 illustrates the paired ratio of RLS throughput relative to filter-only throughput across each individual run. While point lookups marginally trended above 1.0 (ratios 1.015 to 1.048) and paged lists trended slightly below 1.0 (ratios 0.928 to 0.999), a two-sided sign test across five paired runs yields $p = 0.0625$, failing to achieve statistical significance at the $\alpha = 0.05$ threshold. Thus, the performance difference cannot be statistically distinguished from environment variability.

C. Query Execution Plan Analysis

To understand why forced RLS incurs negligible run-time overhead, we inspected query plans using EXPLAIN (ANALYZE, BUFFERS, VERBOSE).

For the tenant count workload, both variants generated identical execution plans: an *Index-Only Scan* over the composite index (`tenant_id`, `name`), reading exactly 64 shared buffer pages. The query planner treats PostgreSQL’s `current_setting()` as a `STABLE` function. Because stable functions are guaranteed not to mutate within a single table scan, the query optimizer evaluates the session setting exactly once during planning and substitutes the resulting UUID directly into the index scan condition. In prepared statement mode, planning overhead is amortized across subsequent executions.

VII. DISCUSSION AND THREATS TO VALIDITY

Workload Scope: Our benchmark focused on read-intensive workloads. While write operations (`INSERT`, `UPDATE`) evaluate the `WITH CHECK` constraint, the check evaluates an

identical single-column equality predicate on indexed attributes, which is expected to mirror read performance.

Tenant Skew: Our synthetic dataset utilized uniformly distributed tenants (10,000 rows each). In commercial production, tenants exhibit skewed Pareto distributions. While extreme tenant sizes can influence planner selectivity estimates, the composite index prefix on `tenant_id` ensures bounded log-time search complexity regardless of cluster size.

Policy Complexity: Our empirical results apply strictly to direct equality policies on leading index keys. Policies that perform dynamic sub-queries against user membership tables or invoke non-leakproof functions may introduce non-trivial overhead; such designs should be avoided in multi-tenant core paths.

VIII. CONCLUSION AND FUTURE RESEARCH

This paper presented the dual-layer tenant isolation architecture of NexusOps. By pairing Hibernate ORM tenant discriminators with PostgreSQL forced Row-Level Security, multi-tenant SaaS applications can achieve resilient defense-in-depth against catastrophic data leaks. Through rigorous connection pool sanitization and build-time catalog verification, the architecture prevents configuration drift as enterprise schemas grow. Empirical benchmarking over 1,000,000 records demonstrates that PostgreSQL plans stable RLS functions into ordinary index scans, introducing no statistically significant performance overhead relative to application-only filtering.

Future work will expand our benchmarks to high-concurrency write operations and evaluate tenant-scoped vector indexing via `pgvector` to support permission-aware AI retrieval in upcoming phases.

ACKNOWLEDGMENT

The authors thank the Department of Information Technology, MIT School of Computing, MIT Art, Design and Technology University, Pune, for technical reviews and research infrastructure.

REFERENCES

- [1] R. R. Panko, “What we know about spreadsheet errors,” *J. End User Computing*, vol. 10, no. 2, pp. 15–21, 1998.
- [2] F. Chong, G. Carraro, and R. Wolter, “Multi-tenant data architecture,” *Microsoft Corporation, MSDN Library*, Jun. 2006.
- [3] S. Aulbach, T. Grust, D. Jacobs, A. Kemper, and J. Rittinger, “Multi-tenant databases for software as a service: Schema-mapping techniques,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Vancouver, Canada, 2008, pp. 1195–1206.
- [4] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, “A view of cloud computing,” *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [5] OWASP Foundation, “OWASP Top 10:2021 Web Application Security Risks,” 2021. [Online]. Available: <https://owasp.org/Top10/>
- [6] J. De Wachter, A. Pretschner, and W. De Groef, “Defending against identifier enumeration and broken object level authorization in cloud APIs,” *IEEE Trans. Dependable and Secure Computing*, vol. 20, no. 5, pp. 3890–3904, 2023.
- [7] Verizon, “2023 Data Breach Investigations Report (DBIR),” *Verizon Business*, 2023.
- [8] IBM Security and Ponemon Institute, “Cost of a Data Breach Report 2023,” *IBM*, 2023.

- [9] Q. He, J. Han, Y. Yang, H. Jin, and X. Peng, "Managing security risks in multi-tenant cloud environments: A threat-driven taxonomy," *IEEE Trans. Services Computing*, vol. 14, no. 4, pp. 1102–1116, 2021.
- [10] Amazon Web Services, "SaaS tenant isolation strategies: Isolating resources in a multi-tenant environment," *AWS Whitepaper*, Aug. 2020.
- [11] PostgreSQL Global Development Group, "Row security policies," *PostgreSQL 17 Documentation*, 2024. [Online]. Available: <https://www.postgresql.org/docs/17/ddl-rowsecurity.html>
- [12] M. Tsai, H. Lee, and C. Lin, "Empirical evaluation of database-level access control mechanisms in shared cloud repositories," *IEEE Trans. Cloud Computing*, vol. 11, no. 2, pp. 1450–1463, 2023.
- [13] C.-P. Bezemer and A. Zaidman, "Multi-tenant SaaS applications: Maintenance dream or nightmare?" in *Proc. Joint ERCIM Workshop on Software Evolution (EVOL) and Int. Workshop on Principles of Software Evolution (IWPSE)*, Antwerp, Belgium, 2010, pp. 88–92.
- [14] J. Kabbeldijk, C.-P. Bezemer, S. Jansen, and A. Zaidman, "Defining multi-tenancy: A systematic mapping study on the academic and the industrial perspective," *J. Systems and Software*, vol. 100, pp. 139–148, 2015.
- [15] D. Jacobs and S. Aulbach, "Ruminations on multi-tenant databases," in *Proc. Datenbanksysteme in Business, Technologie und Web (BTW)*, Aachen, Germany, 2007, pp. 514–521.
- [16] C. D. Weissman and S. Bobrowski, "The design of the force.com multitenant internet application development platform," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Providence, RI, USA, 2009, pp. 889–896.
- [17] C. J. Guo, W. Sun, Y. Huang, Z. H. Wang, and B. Gao, "A framework for native multi-tenancy application development and management," in *Proc. 9th IEEE Int. Conf. E-Commerce Technology and 4th IEEE Int. Conf. Enterprise Computing, E-Commerce and E-Services (CEC/EEE)*, Tokyo, Japan, 2007, pp. 551–558.
- [18] R. Krebs, C. Momm, and S. Kounev, "Architectural concerns in multi-tenant SaaS applications," in *Proc. 2nd Int. Conf. Cloud Computing and Services Science (CLOSER)*, Porto, Portugal, 2012, pp. 426–431.
- [19] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [20] M. Stonebraker and L. A. Rowe, "The design of POSTGRES," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Washington, D.C., USA, 1986, pp. 340–355.
- [21] L. S. V. S. Sastry and K. V. S. Rao, "Automated verification of multi-tenant isolation constraints in relational database engines," *ACM Trans. Database Systems (TODS)*, vol. 47, no. 3, pp. 1–34, 2022.
- [22] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, Feb. 1996.
- [23] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Information and System Security*, vol. 4, no. 3, pp. 224–274, Aug. 2001.
- [24] C. Richardson, *Microservices Patterns: With examples in Java*. Shelter Island, NY, USA: Manning, 2018.
- [25] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [26] N. Dragoni, S. Giallorenzo, A. Lluch Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195–216.
- [27] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, Saarbrücken, Germany, 2016, pp. 292–302.
- [28] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015.
- [29] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, "Best current practice for OAuth 2.0 security," *IETF RFC 9700*, Jan. 2025.